Lab 05: Managing Deployments Using Kubernetes Engine

Develop Your Google Cloud Network | GSP053

Overview

Learned how to manage and scale Kubernetes deployments using multiple deployment strategies including rolling updates, canary deployments, and blue-green deployments.

Objectives

- Create and scale a Kubernetes deployment
- Perform rolling updates, pause, resume, and roll back
- Implement a canary deployment
- Implement a blue-green deployment

Deployment Strategies

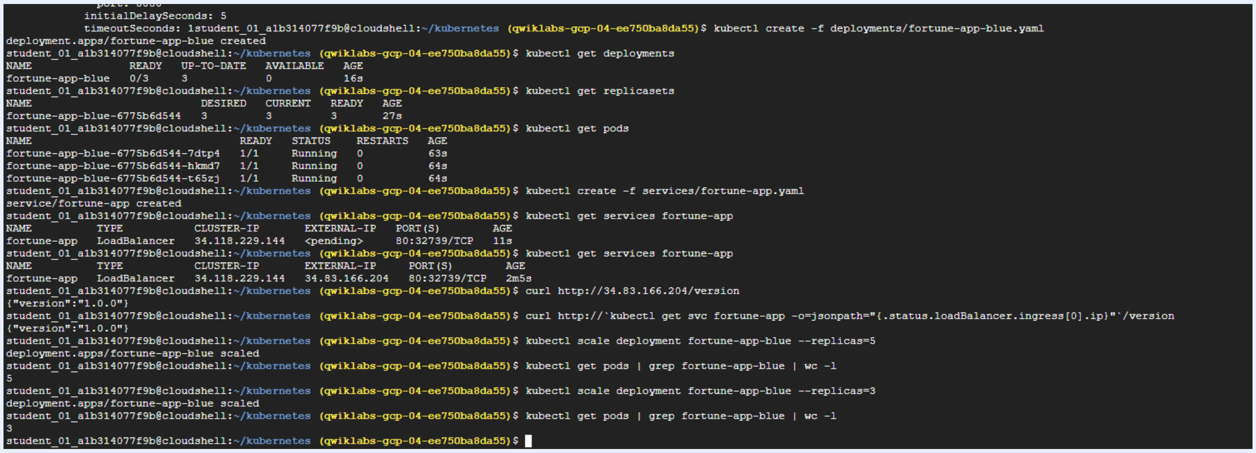
Strategy	How It Works
Rolling Update	Gradually replaces old pods with new version one at a time
Canary	Runs new version alongside old, serving a small subset of traffic
Blue-Green	Switches all traffic instantly from old to new version via service selector

Setup - Create Cluster

```
gcloud container clusters create bootcamp --machine-type e2-small --num-nodes 3 --
scopes "https://www.googleapis.com/auth/projecthosting,storage-rw"
gcloud storage cp -r gs://spls/gsp053/kubernetes . && cd kubernetes
```

Task 1 - Create and Scale Deployment

```
kubectl create -f deployments/fortune-app-blue.yaml
kubectl create -f services/fortune-app.yaml
kubectl scale deployment fortune-app-blue --replicas=5
kubectl scale deployment fortune-app-blue --replicas=3
```

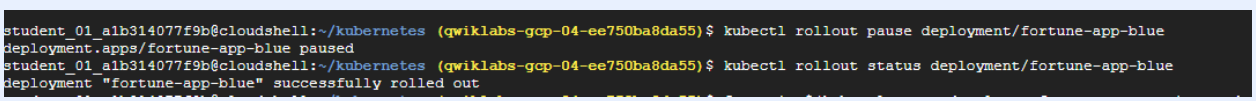


```
            initialDelaySeconds: 5
            timeoutSeconds: 1student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl create -f deployments/fortune-app-blue.yaml
deployment.apps/fortune-app-blue created
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
fortune-app-blue   0/3     3            0           16s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
fortune-app-blue-6775b6d544   3         3         3       27s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
fortune-app-blue-6775b6d544-7dtp4   1/1     Running   0          63s
fortune-app-blue-6775b6d544-hkmd7   1/1     Running   0          64s
fortune-app-blue-6775b6d544-t65zj   1/1     Running   0          64s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl create -f services/fortune-app.yaml
service/fortune-app created
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get services fortune-app
NAME          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
fortune-app   LoadBalancer   34.118.229.144   <pending>     80:32739/TCP   11s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get services fortune-app
NAME          TYPE           CLUSTER-IP       EXTERNAL-IP     PORT(S)        AGE
fortune-app   LoadBalancer   34.118.229.144   34.83.166.204   80:32739/TCP   2m5s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ curl http://34.83.166.204/version
{"version":"1.0.0"}
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ curl http://`kubectl get svc fortune-app -o=jsonpath="{.status.loadBalancer.ingress[0].ip}"`/version
{"version":"1.0.0"}
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl scale deployment fortune-app-blue --replicas=5
deployment.apps/fortune-app-blue scaled
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get pods | grep fortune-app-blue | wc -l
5
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl scale deployment fortune-app-blue --replicas=3
deployment.apps/fortune-app-blue scaled
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get pods | grep fortune-app-blue | wc -l
3
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$
```

Task 2 - Rolling Update

Updated image from 1.0.0 to 2.0.0, paused, resumed, and rolled back the deployment.

```
kubectl edit deployment fortune-app-blue
kubectl rollout pause deployment/fortune-app-blue
kubectl rollout resume deployment/fortune-app-blue
kubectl rollout undo deployment/fortune-app-blue
```



```
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl rollout pause deployment/fortune-app-blue
deployment.apps/fortune-app-blue paused
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl rollout status deployment/fortune-app-blue
deployment "fortune-app-blue" successfully rolled out
```

Task 3 - Canary Deployment

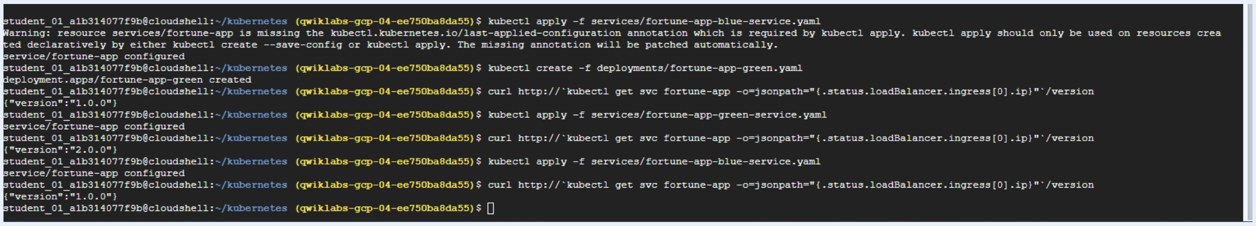
```
kubectl create -f deployments/fortune-app-canary.yaml
```

Running loop showed majority of requests served by v1.0.0 with a small subset from v2.0.0 canary.

```
deployment.apps/fortune-app-canary created
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ kubectl get deployments
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
fortune-app-blue     3/3     3            3           16m
fortune-app-canary   1/1     1            1           8s
student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$ for i in {1..10}; do curl -s http://`kubectl get svc fortune-app -o=jsonpath="{.status.loadBalancer.ingress[0].ip}"`/version; echo;
done
{"version":"1.0.0"}

{"version":"2.0.0"}

{"version":"1.0.0"}

{"version":"1.0.0"}

{"version":"1.0.0"}

{"version":"1.0.0"}

{"version":"2.0.0"}

{"version":"1.0.0"}

{"version":"1.0.0"}

{"version":"1.0.0"}

student_01_a1b314077f9b@cloudshell:~/kubernetes (qwiklabs-gcp-04-ee750ba8da55)$
```

Task 4 - Blue-Green Deployment

```
kubectl apply -f services/fortune-app-blue-service.yaml
kubectl create -f deployments/fortune-app-green.yaml
kubectl apply -f services/fortune-app-green-service.yaml
kubectl apply -f services/fortune-app-blue-service.yaml
```



What I Learned

- Rolling updates gradually replace pods without downtime
- Canary deployments let you test new versions with a subset of real traffic
- Blue-green deployments switch all traffic instantly via service selector update
- kubectl rollout undo rolls back to the previous deployment version
- Scaling with kubectl scale changes the number of pod replicas immediately

Resources

GCP GKE Docs: https://cloud.google.com/kubernetes-engine/docs | Lab ID: GSP053